Parallel Programming

Simple guide: why use it, which tool to pick, and how to run it

Why parallel programming?

A normal program does one step at a time. A parallel program splits the work so many CPU cores (or GPU threads) work together. Result: the same job finishes faster, or a bigger job becomes possible.

- **Faster:** 4 cores can finish a big loop in roughly a quarter of the time.
- **Bigger:** clusters (MPI) and GPUs (CUDA) handle data too large for one core.
- **Careful:** for very small jobs, parallel overhead can make it *slower*.

Which tool should I use?

Tool	Use it when	Compile	Run
OpenMP	One computer, many CPU cores (C/C++ loops)	g++ -fopenmp f.cpp -o f	./f
MPI	Many processes, or many computers (cluster)	mpic++ f.cpp -o f	mpirun -np 4 ./f
Python	Python scripts and data tasks	(not needed)	python3 f.py
Java	Thread pools, server tasks	javac F.java	java F
CUDA	Huge data on an NVIDIA GPU	nvcc f.cu -o f	./f

Key words

- **Process** - a running program with its own memory. **Thread** - a worker inside a process; threads share memory.
- **Shared memory** (OpenMP, threads) - workers use the same data. **Message passing** (MPI) - processes send data to each other.
- **Race condition** - wrong results because threads change the same data at the same time.
- **Speedup** = serial time / parallel time.

Best way to learn

- 1. Write a normal (serial) version first and check the answer.
- 2. Add parallelism to one part only.
- 3. Check the answer is still correct, then measure the time.

Do not mix the tools: plain `g++` does not enable OpenMP or MPI. Use `-fopenmp` for OpenMP, `mpic++` for MPI, `nvcc` for CUDA.

2. Setup: folder and first program

Linux, macOS, WSL on Windows, or Colab (see page 8)

Steps

```
mkdir parallel_lab        # 1. create a folder
cd parallel_lab           # 2. go inside it
code .                    # 3. open it in VS Code
```

4. In VS Code create a file, for example `hello.cpp`, then compile and run it:

```
g++ hello.cpp -o hello    # compile
./hello                   # run
```

Check your tools

```
g++ --version
python3 --version
java -version
```

Install on Ubuntu / Debian

```
sudo apt update
sudo apt install build-essential g++ openjdk-17-jdk python3
sudo apt install openmpi-bin libopenmpi-dev      # only for MPI
```

Test program (plain C++)

```
// hello.cpp
#include <iostream>
int main() {
    std::cout << "Hello from C++!" << std::endl;
    return 0;
}
```

Pattern to remember: `g++ source.cpp -o program` then `./program`. `-o` chooses the name of the executable. On Windows PowerShell run it as `.\hello.exe`.

3. OpenMP

Many CPU cores on one computer

Why use it?

You add one line (`#pragma omp ...`) above a loop or block, and the compiler runs it on several threads. It is the easiest way to use all cores of your CPU in C/C++.

How to run

```
g++ -fopenmp hello_omp.cpp -o hello_omp
./hello_omp                          # uses all cores
OMP_NUM_THREADS=4 ./hello_omp        # use exactly 4 threads
```

Example 1: threads say hello

```
// hello_omp.cpp
#include <iostream>
#include <omp.h>
int main() {
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int total = omp_get_num_threads();
        #pragma omp critical
        std::cout << "Hello from thread " << id
                  << " of " << total << "\n";
    }
    return 0;
}
```

Example 2: parallel loop (sum 1 to 1,000,000)

```
// sum_omp.cpp
#include <iostream>
int main() {
    const long n = 1000000;
    long long sum = 0;
    #pragma omp parallel for reduction(+:sum)
    for (long i = 1; i <= n; ++i) sum += i;
    std::cout << "Sum = " << sum << "\n";
}
```

Output: `Sum = 500000500000` (thread order in Example 1 changes every run - that is normal).

Why reduction? Each thread keeps its own partial sum and OpenMP adds them safely. A plain shared `sum += i` is a race condition and gives wrong answers. Forgetting `-fopenmp` makes the code run on one thread only.

4. MPI

Many processes, one computer or a cluster

Why use it?

Each process has its own memory and a number called its **rank** (0, 1, 2 ...). Processes cooperate by sending messages. Use MPI when one computer is not enough, or to practice cluster-style programming.

How to run

```
mpic++ hello_mpi.cpp -o hello_mpi          # build
mpirun -np 4 ./hello_mpi                   # run with 4 processes
mpirun --oversubscribe -np 8 ./hello_mpi   # more processes than cores
```

Example 1: hello from every process

```
// hello_mpi.cpp
#include <mpi.h>
#include <iostream>
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);   // my number
    MPI_Comm_size(MPI_COMM_WORLD, &size);   // total processes
    std::cout << "Hello from process " << rank
              << " of " << size << "\n";
    MPI_Finalize();
    return 0;
}
```

Example 2: add one value from every process

```
// sum_mpi.cpp
#include <mpi.h>
#include <iostream>
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int local = rank + 1, total = 0;
    MPI_Reduce(&local, &total, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rank == 0) std::cout << "Sum = " << total << "\n";
    MPI_Finalize();
}
```

With 4 processes the output is `Sum = 10` (1+2+3+4).

Remember: `-np 4` (or `-n 4`) = number of processes. Use `--oversubscribe` only if you ask for more processes than CPU cores. On a college/office cluster, follow the admin's instructions.

5. Python

multiprocessing for CPU work, threads for waiting (I/O)

Why use it?

Python's GIL lets only one thread run Python code at a time. So: use **processes** for heavy calculations, and **threads** when the program mostly waits (files, internet).

How to run

```
python3 hello_multiprocessing.py
```

Example 1: process pool (CPU work)

```
# hello_multiprocessing.py
from multiprocessing import Pool, cpu_count

def square(n):
    return n * n

if __name__ == "__main__":          # this line is required
    values = [1, 2, 3, 4, 5, 6]
    with Pool(processes=min(4, cpu_count())) as pool:
        print(pool.map(square, values))
```

Output: `[1, 4, 9, 16, 25, 36]`

Example 2: thread pool (waiting work)

```
# hello_threads.py
from concurrent.futures import ThreadPoolExecutor
import time

def task(n):
    time.sleep(0.2)                 # pretend to wait for a file/network
    return f"Task {n} finished"

if __name__ == "__main__":
    with ThreadPoolExecutor(max_workers=4) as pool:
        for r in pool.map(task, range(4)):
            print(r)
```

Tip: run multiprocessing from a saved `.py` file, not inside a notebook cell - notebooks can hang. For big number arrays, try NumPy first; it is already fast.

6. Java

Thread pools with ExecutorService

Why use it?

An `ExecutorService` keeps a fixed number of worker threads and hands them tasks. It is much easier and safer than creating threads by hand.

How to run

```
javac HelloParallel.java      # compile (creates HelloParallel.class)
java HelloParallel            # run (no .class here)
```

Example

```
// HelloParallel.java
import java.util.List;
import java.util.concurrent.*;

public class HelloParallel {
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(4);
        try {
            List<Callable<Integer>> jobs = List.of(
                () -> 1 * 1, () -> 2 * 2, () -> 3 * 3, () -> 4 * 4);
            for (Future<Integer> r : pool.invokeAll(jobs))
                System.out.println(r.get());
        } finally {
            pool.shutdown();
        }
    }
}
```

Output (one per line): `1 4 9 16`

Rule: the file name must match the public class name (`HelloParallel.java` for `public class HelloParallel`). Always call `pool.shutdown()`, or the program may never exit.

7. CUDA

Thousands of threads on an NVIDIA GPU

Why use it?

A GPU has thousands of small cores - ideal for doing the same operation on huge arrays (images, matrices, machine learning). You need an NVIDIA GPU and the `nvcc` compiler. A CPU-only machine cannot run CUDA.

How to run

```
nvidia-smi                          # is a GPU visible?
nvcc vector_add.cu -o vector_add    # compile
./vector_add                        # run
```

Example: add two arrays on the GPU

```
// vector_add.cu
#include <iostream>
#include <vector>

__global__ void add(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;   // my element
    if (i < n) c[i] = a[i] + b[i];
}

int main() {
    const int n = 8;
    std::vector<float> a(n), b(n, 10.0f), c(n);
    for (int i = 0; i < n; ++i) a[i] = i;

    float *da, *db, *dc;
    cudaMalloc(&da, n * sizeof(float));
    cudaMalloc(&db, n * sizeof(float));
    cudaMalloc(&dc, n * sizeof(float));
    cudaMemcpy(da, a.data(), n * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(db, b.data(), n * sizeof(float), cudaMemcpyHostToDevice);

    add<<<1, 256>>>(da, db, dc, n);                  // 1 block, 256 threads
    cudaDeviceSynchronize();

    cudaMemcpy(c.data(), dc, n * sizeof(float), cudaMemcpyDeviceToHost);
    for (float x : c) std::cout << x << " ";
    std::cout << "\n";
    cudaFree(da); cudaFree(db); cudaFree(dc);
}
```

Output: `10 11 12 13 14 15 16 17`

Flow: copy data to GPU, run kernel, copy result back. For tiny arrays like this one, copying costs more than computing - GPUs pay off only on big data. If `nvcc: command not found`, the CUDA toolkit is not installed (a GPU driver alone is not enough).

8. Run everything in Google Colab

No installation on your PC - each block below is one Colab cell

- Open Google Colab and create a new notebook. For CUDA: **Runtime > Change runtime type > GPU**.
- Lines starting with `!` are shell commands. `%%writefile name` must be the **first line** of a cell; it saves the cell as a file.

C++ and OpenMP

```
%%writefile hello_omp.cpp
#include <iostream>
#include <omp.h>
int main() {
    #pragma omp parallel
    {
        #pragma omp critical
        std::cout << "Thread " << omp_get_thread_num()
                  << " of " << omp_get_num_threads() << "\n";
    }
}
```

```
!g++ -fopenmp hello_omp.cpp -o hello_omp
!OMP_NUM_THREADS=4 ./hello_omp
```

MPI (install once, then build and run)

```
!apt-get update -qq
!apt-get install -y openmpi-bin libopenmpi-dev
```

```
!mpic++ hello_mpi.cpp -o hello_mpi      # after saving hello_mpi.cpp with %%writefile
!mpirun --allow-run-as-root --oversubscribe -np 4 ./hello_mpi
```

Python, Java, GPU

```
!python3 hello_multiprocessing.py       # after saving it with %%writefile
!javac HelloParallel.java
!java HelloParallel
!nvidia-smi                             # GPU check
!nvcc --version                         # CUDA compiler check
```

Colab notes: `--allow-run-as-root` is needed because Colab runs as root (use it only on Colab/personal containers). If the runtime resets, installed packages and files are deleted - save important code to Google Drive.

9. Cheat sheet and common errors

Keep this page for quick revision

Task	Build	Run
Normal C++	g++ f.cpp -o f	./f
OpenMP	g++ -fopenmp f.cpp -o f	./f or OMP_NUM_THREADS=4 ./f
MPI	mpic++ f.cpp -o f	mpirun -np 4 ./f
MPI (more than cores)	mpic++ f.cpp -o f	mpirun --oversubscribe -n 8 ./f
Python	-	python3 f.py
Java	javac F.java	java F
CUDA	nvcc f.cu -o f	./f

Common errors

Problem	Fix
g++: command not found	Install it: sudo apt install build-essential
OpenMP runs on 1 thread / omp.h error	Add -fopenmp when compiling
mpic++: command not found	Install Open MPI: sudo apt install openmpi-bin libopenmpi-dev
MPI: not enough slots	Add --oversubscribe, or lower the -np number
nvcc: command not found	CUDA toolkit missing; use a GPU machine / Colab GPU runtime
Java: class not found	Run javac first; use the class name without .class
Parallel version is slower	Job is too small - use bigger input or fewer workers
Different result each run	Race condition - use reduction or critical (printed order may vary, that is fine)

Official documentation

- Open MPI: docs.open-mpi.org
- OpenMP: openmp.org/specifications
- CUDA: docs.nvidia.com/cuda
- Python: docs.python.org/3
- Java: docs.oracle.com/en/java